

High-Performance Implementation of Multigrid Solver for Lattice QCD

张鑫

中国科学院近代物理研究所

2026 年 9 月 28 日

摘要

传播子求解是格点量子色动力学 (Lattice QCD) 数值计算的核心瓶颈之一。本文介绍 PyQCU 中面向 Wilson-Clover 费米子的 CUDA-C++ 多重网格 (MultiGrid, MG) 实现。方法以奇偶预处理后的 Dirac 算子为细层问题, 通过低模探测、局部正交聚集、转移算子构造和 Galerkin 投射生成递归粗层; Strict 路径采用

$$D_l = X_l + H_l, \quad \hat{D}_l = X_l^{-1} D_l, \quad D_{l+1} = R_l \hat{D}_l P_l,$$

并保留粗层 full geometry 与 $X/Y/\hat{Y}$ 资产。CUDA-C++ 后端结合五流执行、设备端标量、内核融合、CUDA Graph、分块/着色 Galerkin 构造、持久工作区、校验缓存和非阻塞向量 halo, 以降低细层带宽、粗层延迟和分布式通信成本。性能评价严格采用 git tag test27 的最终 capacity-aware 数据: 请求矩阵包含 72 个 combined 单元, 实际完成 66 个精确单元和 132 条 side record; 其余 6 个双 P100 c64 $16 \times 32 \times 32 \times 48$ 请求项因显存或 570 s 时限被显式替代。66 个精确单元的整体中位时间比 $R = t_{\text{QUDA}}/t_{\text{PyQCU}} = 1.025$, MG-2/3 在 35/44 个单元中更快; 若只采用正式 trace-off 口径, MG 为 21/22 个单元有利。全部精确单元均通过独立 full-operator 真残差门, PyQCU 与 QUDA 的最大真残差分别为 7.40×10^{-7} 和 7.97×10^{-7} 。结果支持 “MG 专项性能已达到国际先进实现同量级并具有明显场景优势”, 但不支持将结论外推为全部求解器、全部格点或全部并行拓扑上的普遍领先。

关键词: 格点量子色动力学; Wilson-Clover 费米子; 多重网格; Galerkin 投影; CUDA; MPI; 高性能计算

1 引言

1.1 研究背景

格点 QCD 通过离散化欧几里得时空上的夸克和胶子场, 为非微扰强相互作用计算提供第一性原理框架。在传播子、关联函数和部分子分布函数等计算中, 最耗时的步骤通常是求解具有特殊结构的稀疏复线性系统

$$D(U, m_0)\psi = \eta, \quad (1.1)$$

其中 D 为 Wilson 或 Clover 改进的 Wilson–Dirac 算子, U 为 $SU(3)$ 规范链接, m_0 为格点裸质量, η 为源场。轻夸克质量使 D 接近奇异, Krylov 方法在低频模态上的收敛随格点体积恶化, 即所谓 critical slowing down。多重网格通过显式构造低模粗空间, 将低频误差转移到粗层处理, 从而降低迭代次数对体积的依赖。

1.2 本文范围

本文关注 PyQCU 的 MultiGrid 求解器及其 CUDA–C++ 实现, 内容分为四条主线:

1. 给出 Wilson/Clover 算子、原始 Dslash 和奇偶 Schur 消元的统一数学约定;
2. 说明自适应聚集多重网格的构造, 以及其在 Clover–Wilson 非正规算子上的特化;
3. 分析 CUDA–C++ 后端的并行与内存优化;
4. 基于 test27 最终数据, 对 PyQCU Strict MG 与 QUDA 1.1.0 MultiGrid 进行严格对照, 并明确适用边界。

1.3 证据与口径

本文的性能结论只使用 `git tag test27` 及其 `data/report_multigrid_comprehensive_20260928/final_protocol` 中的原始记录。实现分析同时参考当前 CUDA–C++ 源码和 `docs/report_*.tex` 中的最新技术报告。历史报告中的旧分母或实验性加速比不再作为最终性能结论; 若引用历史版本, 只用于说明实现演进或失败边界。

2 格点 QCD 传播子与 Dirac 算子

2.1 Wilson 作用量和 Dslash

规范链接 $U_{x,\mu} \in SU(3)$ 连接格点 x 与 $x + \hat{\mu}$ 。正向和反向协变平移分别为

$$T_{+\mu}\psi_x = U_{x,\mu}\psi_{x+\hat{\mu}}, \quad T_{-\mu}\psi_x = U_{x-\hat{\mu},\mu}^\dagger\psi_{x-\hat{\mu}}. \quad (2.1)$$

取 Wilson 参数 $r = 1$, 定义无质量尺度因子的跃迁核 H :

$$(H\psi)_x = \sum_{\mu=0}^3 \left[(1 - \gamma_\mu) U_{x,\mu} \psi_{x+\hat{\mu}} + (1 + \gamma_\mu) U_{x-\hat{\mu},\mu}^\dagger \psi_{x-\hat{\mu}} \right]. \quad (2.2)$$

相应的 Wilson 费米子部分和作用量为

$$D_W = (m_0 + 4)I - \frac{1}{2}H = (m_0 + 4)(I - \kappa H), \quad (2.3)$$

$$S_W = \sum_x \bar{\psi}_x D_W \psi_x, \quad \kappa = \frac{1}{2m_0 + 8}. \quad (2.4)$$

因此

$$m_0 + 4 = \frac{1}{2\kappa}. \quad (2.5)$$

PyQCU 求解器中常用的预条件归一化形式为

$$D_{\text{pc}} = I - \frac{\kappa}{u_0} H, \quad D_W = (m_0 + 4) D_{\text{pc}}, \quad (2.6)$$

其中 u_0 为 tadpole 因子。当前 C++ 生产路径等价于 $u_0 = 1$ 。需要强调的是, C++ 原始 Wilson-Dslash 入口返回的是裸核 H , 不含单位项和 $-\kappa$ 因子; 带单位项的 Python 入口则返回 $I - (\kappa/u_0)H$ 。在跨实现比较时, 必须显式保持该归一化差异。

2.2 Clover 改进

Clover 项通过局域四点规范环改进 Wilson 算子的 $O(a)$ 误差。定义有向 plaquette

$$P_{\mu\nu}(x) = U_\mu(x)U_\nu(x + \hat{\mu})U_\mu^\dagger(x + \hat{\mu} + \hat{\nu})U_\nu^\dagger(x), \quad (2.7)$$

以及四叶 Clover 和

$$Q_{\mu\nu} = P_{\mu\nu} + P_{\nu,-\mu} + P_{-\mu,-\nu} + P_{-\nu,\mu}, \quad C_{\mu\nu} = Q_{\mu\nu} - Q_{\mu\nu}^\dagger. \quad (2.8)$$

当前代码级 onsite 修正块为

$$T_p = -\frac{\kappa c_{\text{sw}}}{8u_0} \sum_{\mu < \nu} \gamma_\mu \gamma_\nu C_{\mu\nu}, \quad A_p = I_p + T_p, \quad (2.9)$$

其中有效 $c_{\text{sw}} = 1$ 。预条件归一化和物理归一化算子分别为

$$D_{\text{pc}}^C = I - \frac{\kappa}{u_0} H + T, \quad (2.10)$$

$$D_{\text{phys}}^C = (m_0 + 4)I - \frac{1}{2}H - \frac{c_{\text{sw}}}{16u_0} \sum_{\mu < \nu} \gamma_\mu \gamma_\nu C_{\mu\nu}. \quad (2.11)$$

Clover 项不扩大最近邻 stencil, 但把每个格点的局域自由度由纯颜色空间提升为 $4 \times 3 = 12$ 维 spin-color 空间。其逆或分解必须在细层和粗层算子构造中批量执行。

2.3 奇偶分块与 Schur 消元

将场按格点奇偶性划分为 even 和 odd 两个子空间。归一化 Clover 算子可以写成

$$D_{\text{pc}} = \begin{pmatrix} A_e & -\kappa H_{eo} \\ -\kappa H_{oe} & A_o \end{pmatrix}, \quad A_p = \begin{cases} I_p, & \text{Wilson,} \\ I_p + T_p, & \text{Clover.} \end{cases} \quad (2.12)$$

以奇数子格 Schur 补为例, 消去 even 分量后得到

$$S_o = A_o - \kappa^2 H_{oe} A_e^{-1} H_{eo}, \quad (2.13)$$

$$b_o^{\text{pc}} = b_o + \kappa H_{oe} A_e^{-1} b_e, \quad (2.14)$$

$$x_e = A_e^{-1} (b_e + \kappa H_{eo} x_o). \quad (2.15)$$

对于 Wilson 算子, $A_e = A_o = I$, 因此

$$S_o = I - \kappa^2 H_{oe} H_{eo}. \quad (2.16)$$

该变换是精确消元: 外层 Krylov 求解只作用于半体积未知量, 收敛后再重建另一半。最终正确性仍由完整 Wilson/Clover 算子的真残差判定, 而不是由 Schur 子系统的递推残差代替。

3 多重网格算法与 QCD 特化

3.1 低模粗空间与 Galerkin 投影

多重网格的目标是将误差分解为高频与低频两部分：

$$e = e_{\text{high}} + e_{\text{low}}. \quad (3.1)$$

平滑器负责压低高频误差，粗空间负责表示低频误差。设第 l 层算子为 A_l ，近零模集合满足

$$A_l B_l \approx 0. \quad (3.2)$$

B_l 不是精确核，而是低模空间的数值近似。将 B_l 按 aggregate 重排并在每个 aggregate 内做正交化，得到延拓算子 P_l ，并定义限制算子

$$R_l = P_l^\dagger. \quad (3.3)$$

局部正交化满足

$$P_l^\dagger P_l = I. \quad (3.4)$$

粗层算子则由 Galerkin 投影给出：

$$A_{l+1} = R_l A_l P_l. \quad (3.5)$$

对于非 Hermitian 或非正规算子， $P_l P_l^\dagger \neq I$ ，因此粗空间不能替代细层单位算子。粗层的正确作用是消除 P_l 可表示的低模误差，而不是重新定义一个等价的细层问题。

3.2 格点 QCD 中多重网格为何困难

格点 QCD MG 面临三类相互耦合的约束。

隐式稀疏矩阵不可显式存储。 在代表性较粗层中，若粗自由度为 E ，粗格点数记为 N_c ，则完整矩阵维度为 EN_c ，显式存储需要

$$O((EN_c)^2) \quad (3.6)$$

个复数。对于一个 $N_c \approx 49152$ 、 $E = 24$ 的层级， $EN_c \approx 1,179,648$ ；complex64 稠密矩阵约为 11.1 TB，complex128 约为 22.3 TB。实际实现只能保存有限支撑的 block/stencil，并通过 matrix-free 或打包资产应用算子。

Setup 是高强度数值构造。 粗层资产不仅包含一次矩阵乘法，还包括近零模生成、局部正交化、 P/R 构造、Galerkin probing、onsite 逆和 $\hat{Y}$ 生成。这些步骤涉及多次 full-field matvec、投影、缓存读写和显存峰值控制。若 setup 无法摊销到足够的求解次数，MG 即使减少外层迭代也可能没有端到端收益。

粗层是延迟问题，而非算术问题。 粗层自由度数较小，但每次 V-cycle 都包含多次点积、向量更新、halo 交换和同步。小内核启动、D2H/H2D 标量传输、MPI 延迟和 host/device 同步会主导 wall time。因此，通用 MG 框架即使算法正确，直接移植到 GPU 后也可能无法获得实用加速。

3.3 PyQCU Strict 的 QCD 特化

PyQCU 当前正式主线为 Strict full-coarse MG。其粗层保留完整几何，只在细层使用目标 parity 的 compact 表示。令

$$D_l = X_l + H_l, \quad (3.7)$$

其中 X_l 收集零位移 onsite block, H_l 收集轴邻近 block。左预条件后的算子为

$$\hat{D}_l = X_l^{-1} D_l = I + \hat{H}_l, \quad \hat{H}_l = X_l^{-1} H_l. \quad (3.8)$$

粗层转移满足

$$D_{l+1} = R_l \hat{D}_l P_l = R_l (X_l^{-1} D_l) P_l. \quad (3.9)$$

运行期保存的轴链接资产为

$$Y_{l,\mu}^f, \quad Y_{l,\mu}^b, \quad \hat{Y}_{l,\mu}^f = X_l^{-1} Y_{l,\mu}^f, \quad \hat{Y}_{l,\mu}^b = Y_{l,\mu}^b X_l^{-\dagger}. \quad (3.10)$$

其中 forward 与 backward 的预条件顺序不同。raw Y 主要作为 setup 和诊断数据，常规求解驻留 $\hat{Y}$ 、 X 与 X^{-1} 。

对于细层目标 parity p , R_l 将 compact target-parity 残差映射为 full coarse 向量, P_l 将 full coarse 修正映射回 compact 目标 parity。粗层资产始终保留 full geometry。该约定避免了用 hopping-only 粗算子替代完整算子的错误，也使 MATPC 与递归粗层求解使用一致的物理语义。

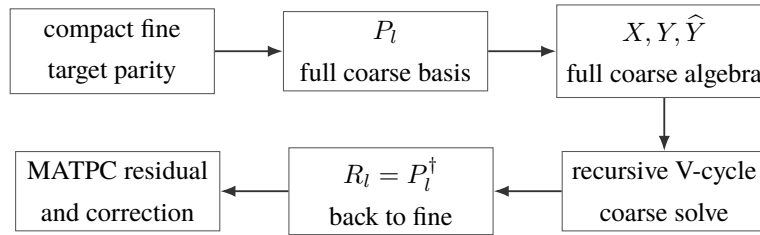


图 3.1: Strict MG 的细层 compact 表示、粗层 full 几何与 $X/Y/\hat{Y}$ 资产之间的数据流。

3.4 V-cycle、平滑器和外层 FGMRES

一次 MG 预条件由细层 MR 平滑、restriction、递归粗层求解、prolongation 和 post-smoothing 组成。固定步 MR 更新为

$$\alpha = \frac{\langle v, r \rangle}{\langle v, v \rangle}, \quad x \leftarrow x + \alpha r, \quad r \leftarrow r - \alpha v. \quad (3.11)$$

外层使用右预条件 flexible GMRES。第 j 个 Arnoldi 列满足

$$z_j = M_j^{-1} v_j, \quad w_j = M_p z_j, \quad (3.12)$$

$$x \leftarrow x + \sum_j z_j y_j, \quad (3.13)$$

其中每个 M_j 可以不同，因此可以使用非固定线性的递归 MG 预条件器。PyQCU 在 restart 边界执行 full-operator 真残差刷新，避免仅依赖递推标量判定收敛。若 Arnoldi 历史长度为 m ，令 B_f 表示一个 compact fine 向量、 B_c 表示一个 full first-coarse 向量，则融合 FGMRES 工作区为

$$B_{\text{work}} = (2m + 5)B_f + 2B_c. \quad (3.14)$$

该工作区在同一几何和 restart 配置下复用于后续求解。

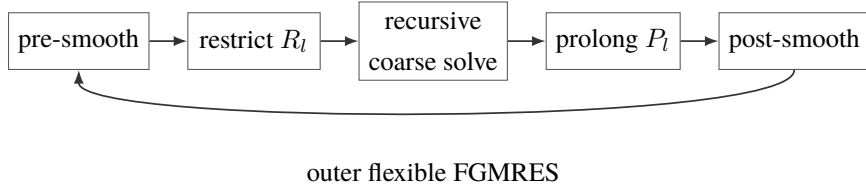


图 3.2: 外层 FGMRES 与内层递归 V-cycle 的嵌套关系。

4 CUDA-C++ 实现与并行优化

4.1 分层软件结构

PyQCU 的 MG 计算路径由四层组成：

1. Python 层负责几何、场、缓存身份和求解器编排；
2. Cython 桥在调用 C++ 前释放 GIL，并使用 params/argv/set_ptrs 三个扁平数组传递状态；
3. C++ 层解析层级几何、创建持久 hierarchy、工作区和通信缓冲；
4. CUDA/MPI 层执行 Dslash、粗层算子、V-cycle、归约和 halo。

Strict 求解的典型生命周期为

$$\text{setup} \rightarrow \text{Strict init} \rightarrow \text{repeated V-cycle/FGMRES} \rightarrow \text{Strict end} \rightarrow \text{release.} \quad (4.1)$$

相较于 legacy 路径，Strict 路径在全部调用间保持 per-instance `_SET_INDEX_` 不变，并复用同一 hierarchy。

4.2 细层内核与访存

Wilson Dslash 单次应用读取四个正向和四个反向邻居。对于每个方向， $1 \pm \gamma_\mu$ 将四分量旋量投影至两个独立分量，因此 CUDA 内核可以先做 spin projection，再执行 $SU(3)$ 矩阵乘法，降低寄存器和内存压力。时空维置于张量最后四个轴，使相邻格点在线程块内连续，有利于合并访存。

Clover 项不改变最近邻支持，但要求每个格点读取和应用 12×12 onsite 块。细层实现批量构造 Clover、批量求逆并将 D_{ee}^{-1} 或其他 onsite 因子复用于算子应用和 Schur 消元。该设计避免逐点调用通用矩阵逆，并使 Dslash 的主体保持带宽受限而不是控制流受限。

4.3 粗层求解与主机同步

粗层向量较小，逐迭代 cublas 调用、D2H/H2D 标量传输和 cudaStreamSynchronize 往往比算术本身更昂贵。PyQCU 采用以下措施：

- 将 $\alpha, \beta, \omega, \rho$ 等标量保留在设备端 device_vals，减少逐步骤主机往返；

- 将 β -更新与 ρ 保存、 r/x 更新等元素操作融合；
- 对固定粗层迭代段使用 CUDA Graph，减少内核启动开销；
- 对小最粗层尝试 cooperative fused solve，当向量过大或多 rank 条件不满足时回退到逐迭代路径；
- 在单 rank 快速路径中增大残差检查间隔，只在稳定检查点读取主机标量。

这些优化的共同目标是把粗层从 “tiny-kernel latency dominated” 转变为可摊销、可预算的执行阶段。

4.4 Setup、显存与缓存

Galerkin 构造按粗基向量探测 $P_l^\dagger A_l P_l$ 的列。直接逐列构造会产生大量 full-field 调用。当前实现采用着色和 site/column 批处理：互不冲突的 source 在同一个 batch 中执行，并根据 workspace 上限自动选择批大小；当显存不足时回退到更保守的 batch。

Strict runtime cache 使用 HDF5 保存 blocked basis、onsite inverse、 $\hat{Y}$ 和必要元数据。缓存 identity 包含格点几何、精度、层级、blocking 和算子参数；每个 tensor 记录 shape、dtype 和 SHA256。读取前逐 tensor 校验，写入使用临时文件与原子发布。该机制避免重复执行小时级 setup，同时防止身份不匹配或内容损坏被静默复用。

4.5 分布式粗层通信

在多 rank 模式下，每个 rank 只持有本地粗块。粗层 halo 只需交换活跃 face，而不是执行完整全量 gather。c64 分布式 vector halo 使用以下流水线：

$$\text{active-face pack} \parallel \text{local base kernel} \rightarrow \text{nonblocking MPI} \parallel \text{local tail} \rightarrow \text{remote correction.} \quad (4.2)$$

输入就绪 event 由 main stream 记录，交换流完成 pack、pinned-host staging 和非阻塞 MPI 收发。main stream 先计算不含远端项的局部核，随后由合并的 correction kernel 加入 remote forward 与 backward 贡献。

该路径改变浮点加法顺序。complex64 分布式路径默认启用 overlap；complex128 默认关闭，只有显式设置 PYQCU_STRICT_OVERLAP_C128=1 并重新通过大格真残差和迭代回归后才允许启用。当前 link halo 仍使用阻塞式 Sendrecv，但通过 pointer identity 缓存；device-aware MPI 与 NVSHMEM 路径属于后续优化范围。

5 性能评价

本章的矩阵口径、覆盖统计、时间比、阶段记录、残差、显存和容量替代结论，均以 docs/report_multigrid_quda_pyqcu_20260928.tex 及其对应的 test27 原始数据为主要依据；算子构造和实现机制部分再结合当前 CUDA-C++ 源码与其他技术报告。

5.1 实验协议

性能评价采用 QUDA 1.1.0 MultiGrid 作为对照。设备包括单卡 Tesla V100-SXM2-32GB (sm_70) 和双 Tesla P100-PCIE-16GB (sm_60, process grid 2x1x1x1)。矩阵覆盖:

- 精度: complex64 与 complex128;
- 求解: BiCGStab reference、MG-2、MG-3;
- trace: on 与 off;
- c64 格点: $8^3 \times 16$ 、 16^4 、 $16 \times 32 \times 32 \times 48$;
- c128 格点: $8^3 \times 16$ 、 16^4 、 $16 \times 16 \times 32 \times 32$ 。

每个精确 side case 执行 1 次 cold、2 次 warmup 和 5 次 steady, 性能统计使用 steady median。配置哈希和输入 bundle 必须一致, 最终通过独立 full-operator 真残差判定:

$$\frac{\|b - Dx\|}{\|b\|} < 5 \times 10^{-6}. \quad (5.1)$$

单实例调度上限为 570 s, 单设备显存峰值门限为设备容量的 85%。trace-on 包含诊断同步开销, 只用于阶段归因; 正式性能结论使用 trace-off。

表 5.1: 最终矩阵覆盖与公平性门禁

项目	观察值	要求
请求 combined 单元	72	计划矩阵
精确 combined 单元	66	66/66 通过
精确 side record	132	132/132 通过
1 cold / 2 warmup / 5 steady	132/132	全部完整
config hash 一致	66/66	全部一致
input bundle 一致	66/66	全部一致
真残差通过	66/66	全部 $< 5 \times 10^{-6}$
trace-on MG 曲线	44/44	全部存在

5.2 总体结果

定义时间比

$$R = \frac{t_{\text{QUDA,steady}}}{t_{\text{PyQCU,steady}}}. \quad (5.2)$$

$R > 1$ 表示 PyQCU 更快。66 个精确单元的整体中位数 $R = 1.025$, PyQCU 在 35 个单元中有利、31 个单元中较慢。MG-2/3 共 44 个单元, 其中 35 个有利; 若只取正式 trace-off 口径, MG 为 21/22 个单元有利。相反, BiCGStab reference 为 0/22, QUDA 全部更快。因此本文只主张 MG 场景优势, 不将其外推为 PyQCU 全部求解器的优势。

5.3 设备与精度分组

V100 上的 MG 优势最明显, V100 c64 MG-3 与 c128 MG-3 的分组中位时间比分别为 4.74 和 3.22。双 P100 的 MG-2/3 中位数均大于 1, 但幅度较小。分组极值表明结果对格点体积和残差轨

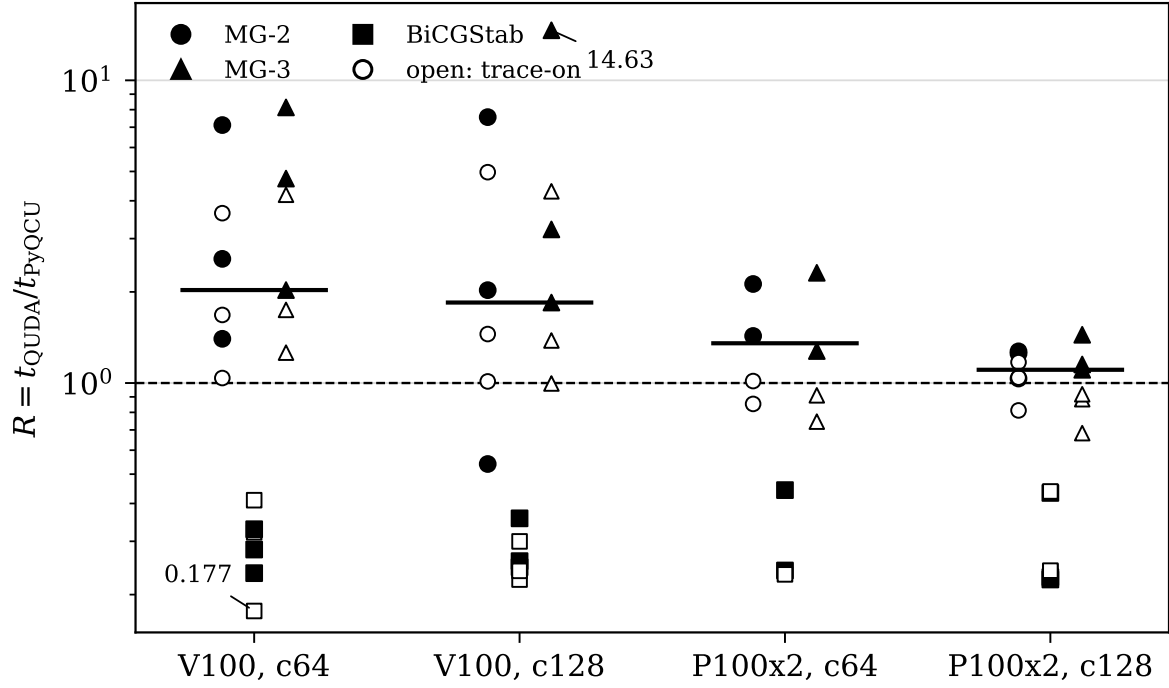


图 5.1: 66 个精确单元的逐点时间比。实心符号为 trace-off，空心为 trace-on；水平短线为对应设备-精度组的 trace-off 中位数。极值是单点，不表示置信区间。

表 5.2: trace-off 分组中位时间比 R

设备/精度	MG-2	MG-3	BiCGStab	备注
V100 / c64	2.5717	4.7366	0.2818	$n = 3$
V100 / c128	2.0269	3.2174	0.2580	$n = 3$
P100×2 / c64	1.7773	1.7944	0.3416	$n = 2$
P100×2 / c128	1.2502	1.1517	0.2284	$n = 3$

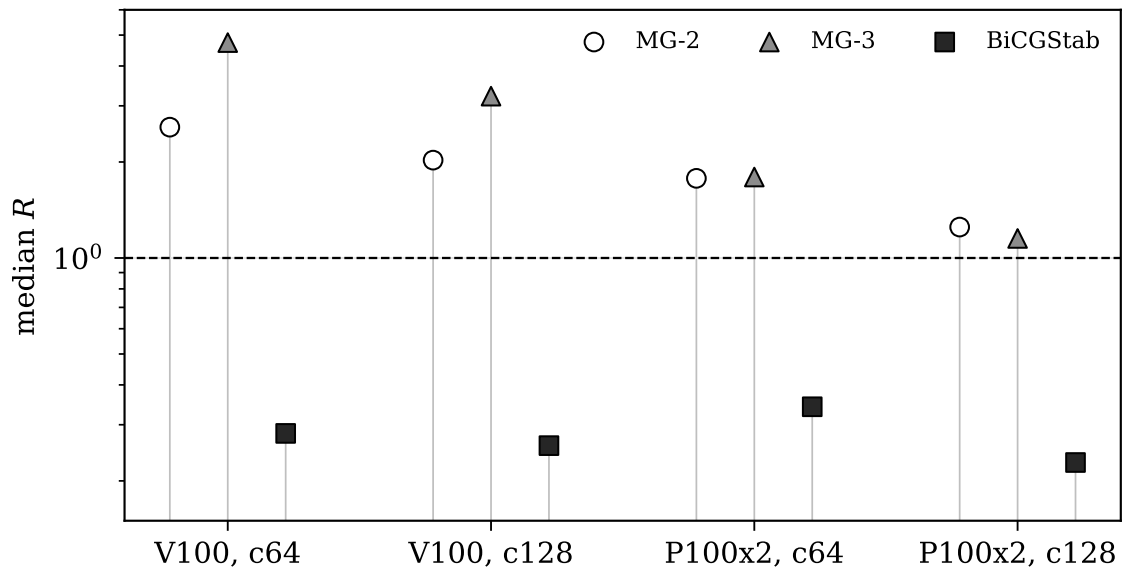


图 5.2: trace-off 条件下各设备、精度和求解器的中位时间比。

迹敏感：V100 c128 MG-3 的最大单点为 14.63，而 V100 c128 MG-2 中存在 0.54 的慢点。因此不能以单一最大值代表设备性能。

5.4 阶段归因与容量边界

表 5.3: trace-on MG 阶段耗时份额中位数

实现	pre	post	restrict	prolong	coarse	other
PyQCU	2.1%	2.0%	0.5%	0.5%	43.9%	50.9%
QUDA	10.5%	17.3%	1.0%	0.7%	69.3%	0.1%

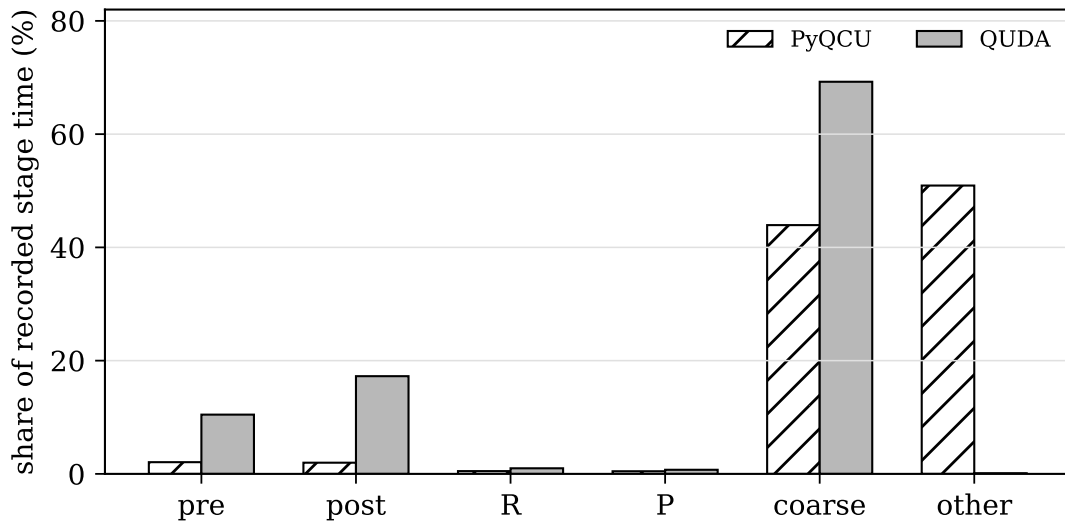


图 5.3: recorded stage time 的中位数份额。两侧层级映射不同，只能比较各自的总时间结构，不能把同名阶段跨实现直接相加。

PyQCU 的 trace 记录中，coarse solve 与 other 合计约占 95%；QUDA 的主要成本集中在具名 coarse 阶段。由于两侧层级映射不同，other 与同名列不能直接跨库比较。该结果说明后续优化应首先聚焦粗层求解、同步、残差检查和通信，而不是继续优化细层算术。

双 P100 的 c64 $16 \times 32 \times 32 \times 48$ 请求项未进入精确矩阵。三次尝试为：默认 overlap 路径峰值约 15.9 GB，占设备容量 93.1%；PYQCU_MPI_OVERLAP=0 为 14.8 GB，占 86.2%；extended CPU-staging 降至 12.4 GB，占 72.4%，但 MG-2 setup 仍超过 570 s。对应 6 个 combined 项被显式替代为 16^4 ，并单独记录，未伪装成原格点性能。

5.5 残差与显存

表 5.4: 精确矩阵的最大 full true residual 与 device-wide 峰值显存

实现	样本	最大真残差	最大观测显存
PyQCU	66	7.40×10^{-7}	11907.3 MiB
QUDA	66	7.97×10^{-7}	23063.3 MiB

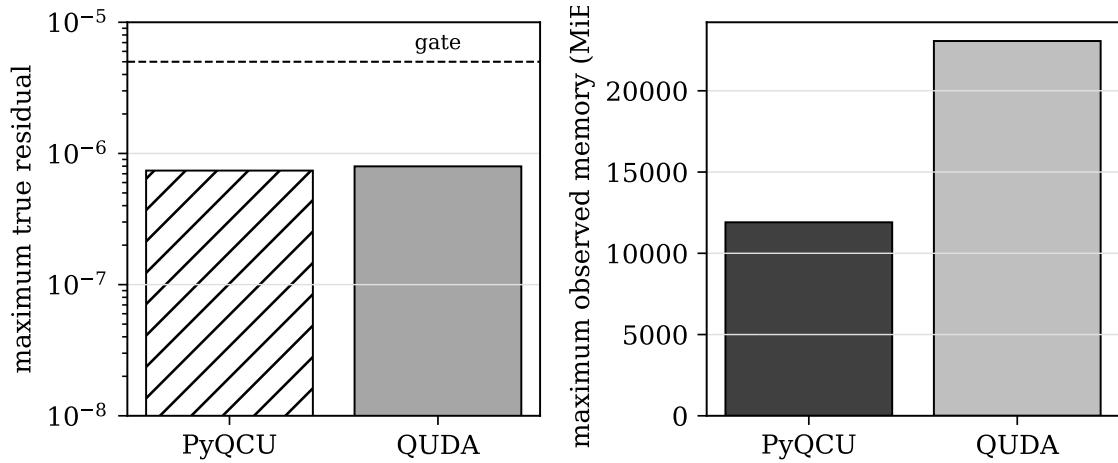


图 5.4: 最大真残差与 device-wide 最大观测显存。显存 sampler 可能包含同设备上的其他进程，因此用于容量门限检查，不声明为进程独占峰值。

两种实现的最大真残差均远低于 5×10^{-6} 门限。保留样本的最大 device-wide 显存均低于 85% 门限，其中 PyQCU 的最大观测值约为 QUDA 的一半。该比较必须结合设备和格点分组理解，不能单独推断所有配置下的内存效率。

6 讨论

6.1 优势

1. **MG 专项性能明确。**在 66 个精确单元中，MG-2/3 为 35/44 有利；trace-off MG 为 21/22 有利。
2. **V100 场景优势显著。**V100 c64/c128 MG-3 trace-off 分组中位比分别为 4.74 和 3.22。
3. **结构扁平、自主可控。**求解器、Cython 桥、C++ ABI、CUDA 内核和测试矩阵位于同一仓库，便于针对 Clover-Wilson 迭代和增加新功能。
4. **资源与正确性证据完整。**配置哈希、输入 bundle、1/2/5 phase、阶段时间、真残差、显存和容量替代均保留原始记录。

6.2 局限

1. 66 个精确单元的总中位仅为 1.025，整体优势较弱。
2. BiCGStab reference 为 0/22 有利，说明优势来自 MG，而不是所有线性求解器。
3. trace-on 引入明显同步开销，部分双 P100 分组退到 1 以下。
4. 双 P100 c64 大格触发显存或 570 s setup 限制，必须使用显式容量替代。
5. complex128 distributed overlap 改变浮点累加顺序，默认关闭；link halo 仍使用阻塞通信。
6. 当前证据主要覆盖 V100 单卡和双 P100，不能外推到任意拓扑或千卡级 MPI。

6.3 后续工作

后续工作按优先级分为四类：

1. 将 coarse solve、残差检查和全局归约的同步进一步合并；
2. 扩展 device-aware MPI、持久 request 和可选 NVSHMEM 通信；
3. 降低 setup 峰值显存，使大格 c64/c128 可进入正式矩阵；
4. 将稳定的 Python/ABI 回归扩展到更多作用量、混合精度和国产加速器后端。

7 结论

本文给出了 PyQCU 中面向 Wilson–Clover 费米子的高性能 MultiGrid 实现，并从算子定义、低模粗空间、Galerkin 投影、CUDA–C++ 优化和性能对照五个层面进行了分析。Strict 路径使用 $D = X + H$ 、 $\hat{D} = X^{-1}D$ 和 $D_{l+1} = R\hat{D}P$ ，在保持完整 Clover 物理语义的同时，将低模误差转移到粗层。CUDA–C++ 后端通过矩阵自由/打包资产、设备端标量、流重叠、融合内核、图执行、显存预算、原子缓存和非阻塞 halo 降低真实 wall time。

test27 的最终数据说明：

1. 66 个精确单元全部通过 config、input 和真残差门；
2. MG-2/3 在 35/44 个单元中优于 QUDA，trace-off MG 为 21/22；
3. V100 上的 MG 优势最明显，双 P100 在 trace-off 下总体有利；
4. 未完成的 6 个双 P100 c64 大格请求项被明确替代，没有混入公平加速比；
5. PyQCU 的最大真残差和最大观测显存均满足预设门限。

因此，当前实现可作为一个结构扁平、自主可控并且在典型 MG 场景接近国际先进实现的求解器原型。下一阶段的决定性问题是粗层同步、通信重叠、setup 显存和更大规模拓扑，而不是继续扩大小格或单点加速比。

A 复现接口

```
RUNNER=pyqcu/testing/qcu/strict/quda_comparison
source ./env.sh
bash ./build.sh
bash ./install.sh
```

```
python -B "${RUNNER}/run_strict_fast.py" \
    --fail-fast --json strict-fast.json
```

```
mpirun -np 2 python -B \
    "${RUNNER}/strict_mpi_solve_probe.py" \
    --shape 8 8 8 16 --grid 2 1 1 1 --dtype c64 \
    --galerkin-mode colored
```

```
python -B "${RUNNER}/bench_mg_matrix_full.py" \
--list
```

正式矩阵、侧记录、聚合和报告生成入口分别为 `bench_mg_matrix_full.py`、`bench_strict_vs_quda.py`、`assemble_final_matrix.py` 和 `build_mg_report.py`。

B 原始证据索引

表 B.1: 最终性能证据文件

文件	内容
<code>units.csv</code>	66 个精确单元的总时间、迭代数、真残差和显存
<code>stages.csv</code>	逐层 <code>pre/post/R/P/coarse/other</code> 阶段记录
<code>references.csv</code>	BiCGStab cold/warmup/steady 原始样本
<code>requested_matrix_status.csv</code>	72 个请求单元的精确或替代状态
<code>capacity_substitutions.csv</code>	6 个双 P100 c64 大格替代记录
<code>summary_analysis.json</code>	覆盖、分组和残差汇总
<code>mg_memory.csv</code>	逐 side 显存原始记录
<code>mg_residual_curves.pdf</code>	trace-on 收敛曲线
<code>mg_stage_breakdown.pdf</code>	逐层阶段堆叠图

参考文献

- [1] K. G. Wilson, Confinement of quarks, *Physical Review D* **10**, 2445 (1974). <https://doi.org/10.1103/PhysRevD.10.2445>
- [2] B. Sheikholeslami and R. Wohlert, Improved continuum limit lattice action for QCD with Wilson fermions, *Nuclear Physics B* **259**, 572 (1985). [https://doi.org/10.1016/0550-3213\(85\)90002-1](https://doi.org/10.1016/0550-3213(85)90002-1)
- [3] R. Babich, J. Brannick, R. C. Brower, M. A. Clark, T. A. Manteuffel, S. F. McCormick, J. C. Osborn, and C. Rebbi, Adaptive multigrid algorithm for the lattice Wilson–Dirac operator, *Physical Review Letters* **105**, 201602 (2010). <https://doi.org/10.1103/PhysRevLett.105.201602>
- [4] M. A. Clark, R. Babich, K. Barros, R. C. Brower, and C. Rebbi, Solving lattice QCD systems of equations using mixed precision solvers on GPUs, *Computer Physics Communications* **181**, 1517 (2010). <https://doi.org/10.1016/j.cpc.2010.05.002>

- [5] QUDA Collaboration, QUDA: A library for lattice QCD computations on GPUs. <https://github.com/lattice/quda>
- [6] M. Rottmann et al., DD- α AMG: Adaptive algebraic multigrid for lattice QCD. <https://github.com/mrottmann/DDalphaAMG>
- [7] E. Weinberg, quantum-mg: Multigrid solver package for lattice QCD. <https://github.com/weinbe2/quantum-mg>
- [8] 张鑫, PyQCU: Python/Cython and CUDA-C++ lattice QCD toolkit. <https://github.com/zhangxin8069/PyQCU>